



Domain Knowledge Belongs In The MCP Server

Seven months of production MCP at a 350-person company

David Golverdingen · Senior Engineer & MCP Architect, Warmtebouw
VibeKode Netherlands · 7 October 2026

Someone who has never opened our ERP
is going to ask it about **some data**.

What is going to tell the agent what it means?

✓ **Transport**
solved for everyone

✓ **Auth**
solved for everyone

? **Meaning**
different at every company,
every tool, every field

Most MCP servers are **empty of meaning**.

“Is this building at risk of overheating?”

one building in Rotterdam · one field
temperatuuroverschrijding: **3.59** *temperature exceedance*

what the model gets

unit *none given*
limit *none given*



reads as ?

| 3.59 hours a year
| 3.59 degrees

MODEL GUESS

“**Low risk**, well below the 40-hour standard.”

plus one line

unit *none: an indicator*
limit *above 1.5 = significant*



means

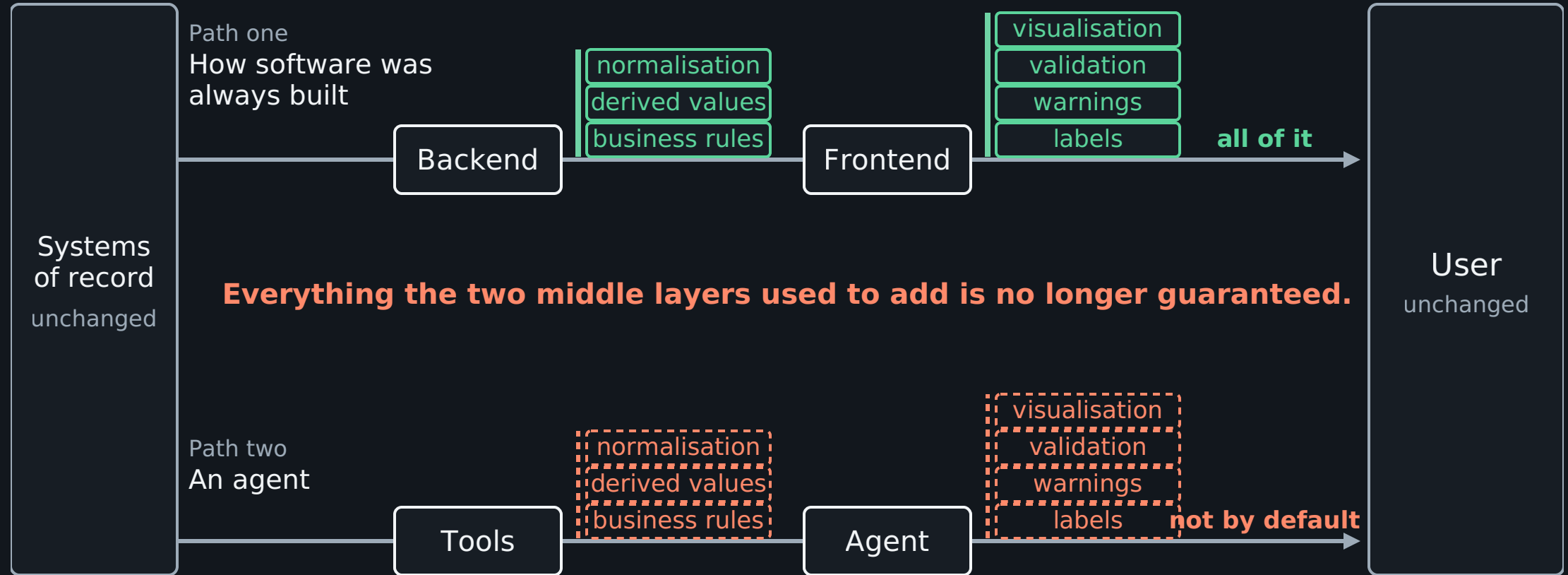
| **over twice the limit**

GROUNDING

“**Significant risk**: 3.59 against a limit of 1.5.”

3.59. Nothing in the response says of what.

Where did the meaning go?

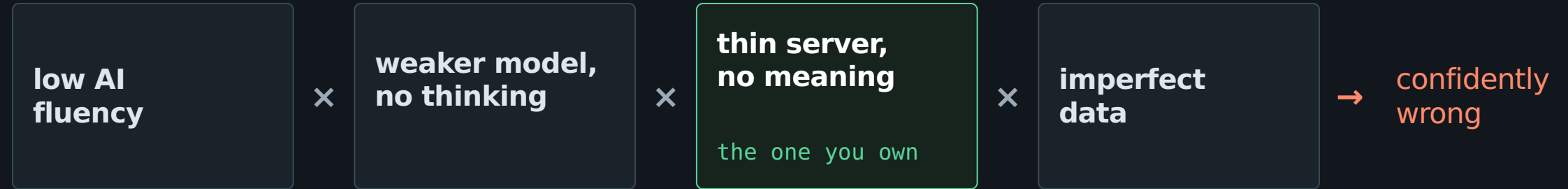


The API could be dumb because the application was smart.

**The agent isn't empty.
The server is.**

“It works” may simply mean **the model guessed correctly.**

Why it works for you, and not for them



pilots call this the error chain

Break **one** link and the odds drop.

WARMTEBOUW.



16

custom MCP servers in production, mostly non-developers

103

tools we wrote the descriptions for

8

MCP Apps: charts, tables, maps, forms, and one game

ERP

BIM

Estimating

Building automation

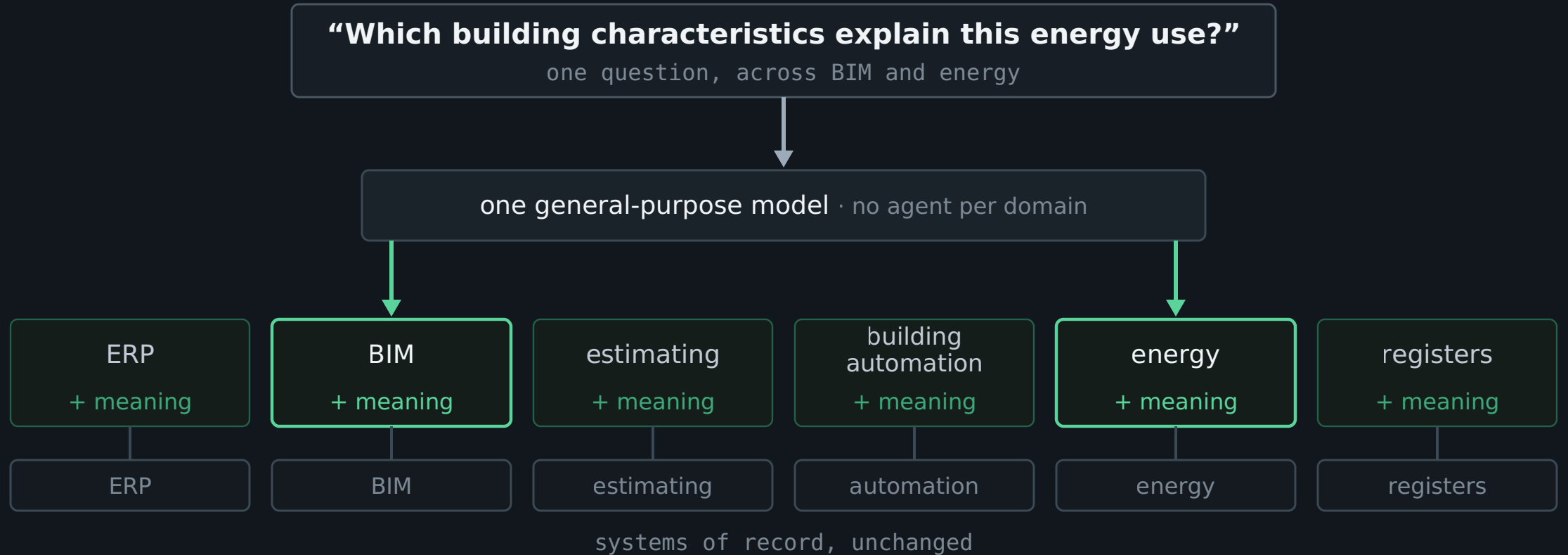
Energy

External registers

IT

350-person Dutch mechanical building services contractor.
Five developers.

We scaled capabilities, not agents



the meaning lives once, in the capability that owns the data

each one carries access, meaning and bounded authority, so any question can cross them

If the interface is empty, **an agent on top of it is a guess with a job title.**

Who discovers the meaning?

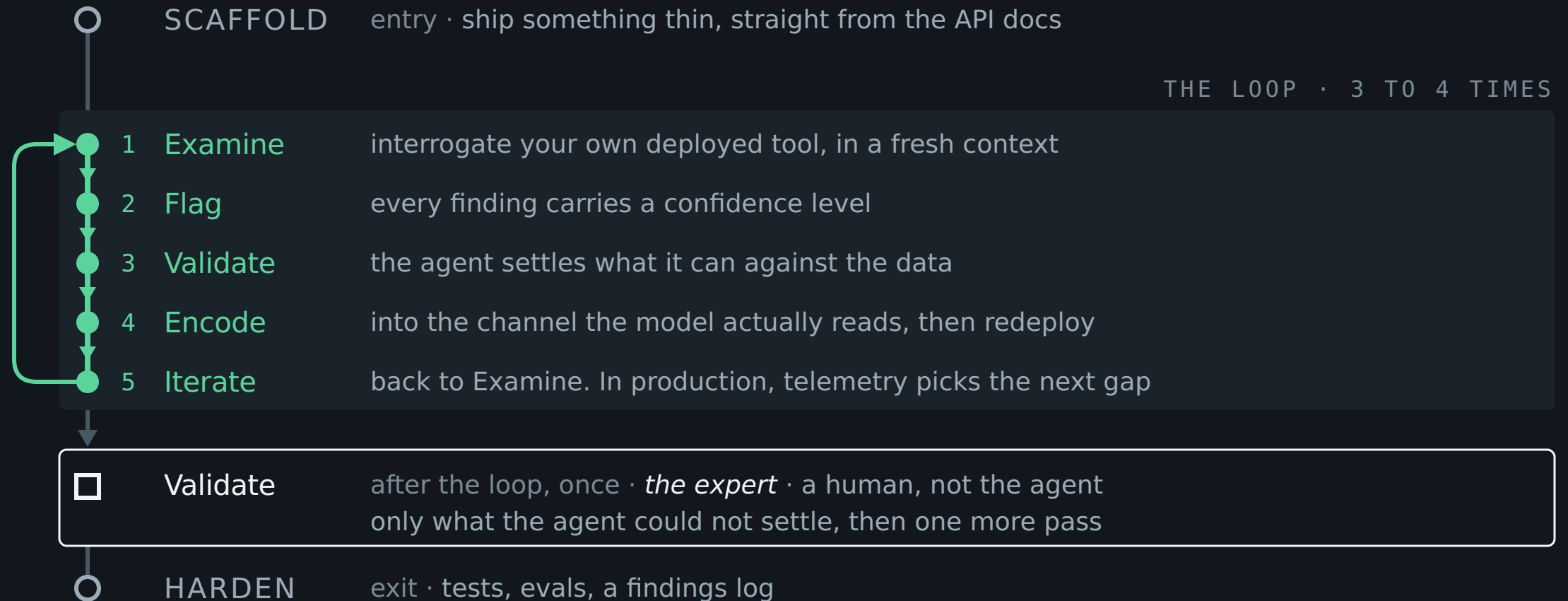


THE SIX LEVELS

L1 · API wrapper	nobody. The agent reconstructs the domain itself
L2 · Descriptive tool	the author, briefly
L3 · Domain-aware	human in the lead
L4 · Self-teaching	the agent, from the real data <i>then: putting back the rest of what the application did</i>
L5 · Interactive	how it is shown · MCP Apps
L6 · Safe write	how it is changed · guarded mutation

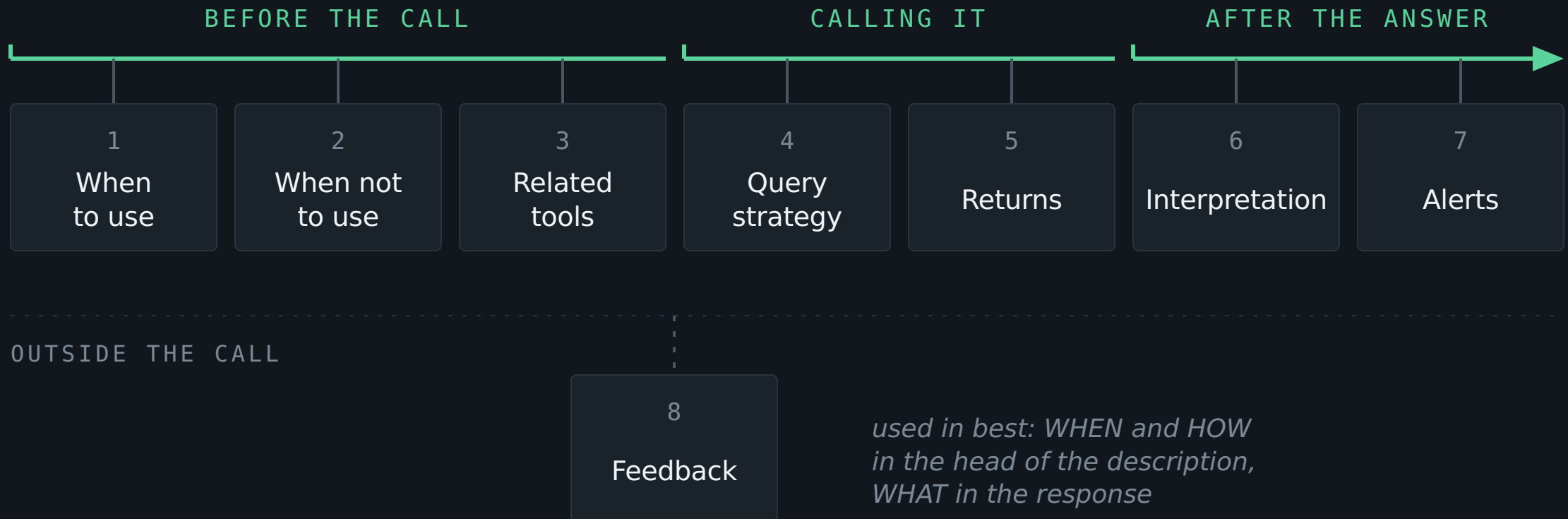
Level 4 is not more work than level 3. *It's less, and better grounded.*

Introspective Context Engineering for MCP



The spec gives you the shape. The data gives you the meaning.

Every tool, same eight blocks



Three of those groups are needed at **three different moments**.

What that looks like in the shared repo

`get_building_profile` in `best` · shortened from `src/tools/get-building-profile-best.ts`

- 1 **WHEN TO USE:** what building is at a Dutch address: label, bouwjaar, area, and label-based estimates (CO₂, space-heating gas, heat-pump readiness, overheating).
- 2 **WHEN NOT TO USE:** this server has NO metered energy consumption: no meter readings, no actual gas or electricity use. Not for addresses outside the Netherlands.
- 3 **RELATED TOOLS:** `get_weather_context(latitude/longitude = coordinaten.lat/lon)` for degree days and weather correction.
- 4 **QUERY STRATEGY:** postcode = 4 digits + 2 capitals, no space ("3543AR"). huisnummer = integer only; a letter goes in huisletter (28A → 28 + "A"), an addition in toevoeging. Several units match? Retry with ...
- 5 **RETURNS:** interpretation, derived, candidates, then BAG facts (bouwjaar, oppervlakte_bag_verblijfsobject_m2 = ONE unit, coordinaten) and the EP-Online label (energielabel, berekeningstype, *_berekend_* ...
- 6 **INTERPRETATION:** read `interpretation` FIRST: this record's computed values and reading rules. Quote them; do not recompute. For every record:
 - **CALCULATED vs MEASURED:** every EP-Online energy figure is calculated by the label method, never measured. Paris Proof and other metered benchmarks are defined on measured final energy, so ...
- 7 **ALERTS:** interpretation.alerts: computed verdicts and this record's branch (not found, several units, no label).

One paragraph per block, in the same order, in every tool.

How do I know it's right?

Expert validation

the AI interviews the expert
on what it flagged ~10% of the metadata

Full review

the expert reads the finished text,
end to end

Evals

questions with a known answer, scored on every version
the right chart, refused by the schema: 54 calls the log never saw

Field validation

the people whose data it is

You review. You don't author.

Every call carries what it was **trying to find out**

“fault reports on the air handling unit, to judge whether the failures recur”

“whether a quote was raised after the inspection for the defective boiler”

“counting the subscription lines before fetching the full records”

“searching on description, because the project number returned no match”

“checking whether the create committed, before retrying, to avoid a duplicate”

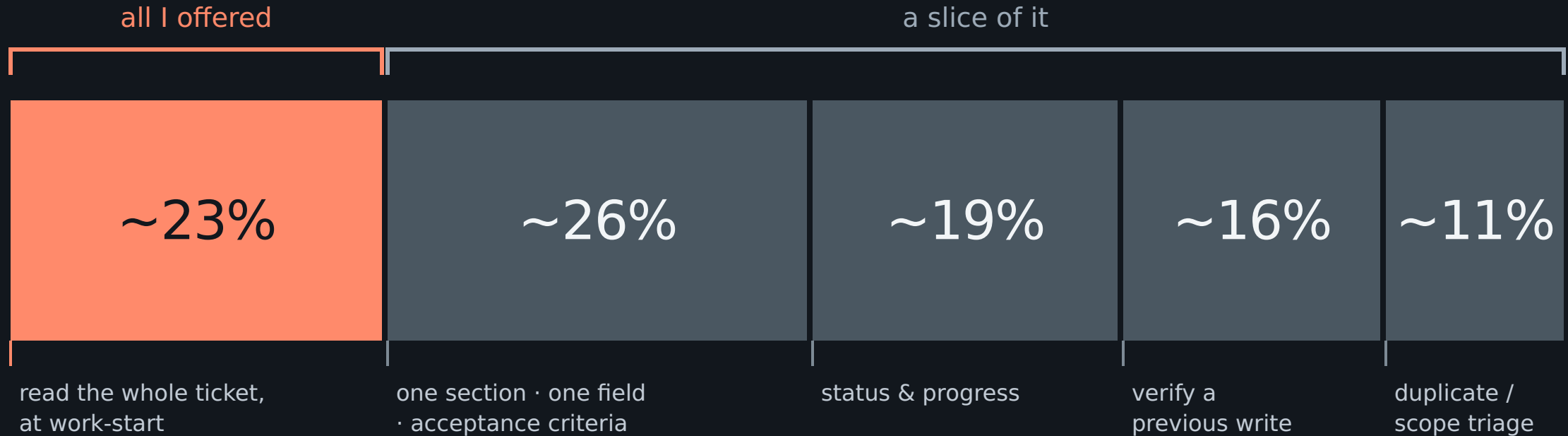
“redacted (special-category server)”

one field: `queryIntent`

A log of calls becomes
a log of questions.

I built `get_ticket` to return a ticket

queryIntent, measured · classified with AI



They only asked for the whole thing **because the whole thing was all I offered.**

What the feedback loop changed

Selective retrieval	one ticket, read twice, 24 seconds apart: 4 KB vs 265 KB. roughly 66x less, same information.	capability
Summaries	the server computes them, instead of shipping rows for the model to add up.	capability
Alerts	fire only when the condition applies. Conditional meaning, at zero catalog cost.	a sentence
Derived values	the meaning arrives as a value, not as a warning.	capability
The write reports back	16% of calls were verifying a previous write. Now the write says what it did, and the call is gone.	capability

Four changed what the tool can **do**. **One** was something I wrote.

Not prompt tuning. **Agent interface engineering**.

Then I measured it

13

of 23

of my first predictions
were wrong

runs	6,000+ scored runs · Claude Haiku, Sonnet and Opus
before	every prediction registered before its run
after	server log reconciled with the agent's own trace
servers	same data: thin · rich , my first version · best

one author · one domain · one model family · evals on Claude Code · delivery on five clients

My intuition was wrong **more often than it was right.**

What held up: capabilities

Do the math in the server	a gas estimate full description, the model does the math · the server computes it	1/20 → 20/20
Include the data the rule needs	same rule + reference-period degree-days score is Haiku · Sonnet, Opus: already right, 1 call instead of 7	2/20 → 15/20
Let the schema say it	apartment 28A: the thin schema has no house letter thin Opus refuses · Haiku with the parameter finds the apartment	0/18 → 18/18
Refuse, with the fix	a table header drops “calculated” after a warning: none redrawn · refused: corrected, all drawn	2/10 → 10/10

Once the server does the deterministic work, **every model saturates on it.**

per model: guidance helps the middle one; a missing fact beats every model

On these tasks, the weakest model with the layer **beat the strongest without it.**

Better was also cheaper

The whole layer	best against rich, same questions	cheaper in 30 of 32
One line about 3.59	181 characters added to the response without it, every run invented a unit	–15% tokens
Ship the data	Sonnet, Opus: 1 call instead of 4 to 16	–86% calls
Leaner schemas	chart and table tools, same reliability	–21 to –29%
Not free	five alerts the question did not need	+1,172 tokens

The expensive part is not the metadata. **It is the guessing.**

Wrong is worse than missing

volume does not hurt: two thirds of the guidance cut, **30/30** → **30/30** · one rule among 100, found as often

IS THIS OFFICE ON TRACK FOR PARIS PROOF 2040?

one plausible line in my
metadata

... Paris Proof kantoor: 70 kWh/m²

59/60 wrong

one sentence added

CALCULATED, not MEASURED

0/60 → 59/60

WHICH HALF OF THAT SENTENCE DID THE WORK? THREE MODELS

the fact only

label figures are calculated; Paris Proof is measured

25/30

the instruction only

"say it cannot be compared", with no reason

10/30

both

the fact, and what to do with it

30/30

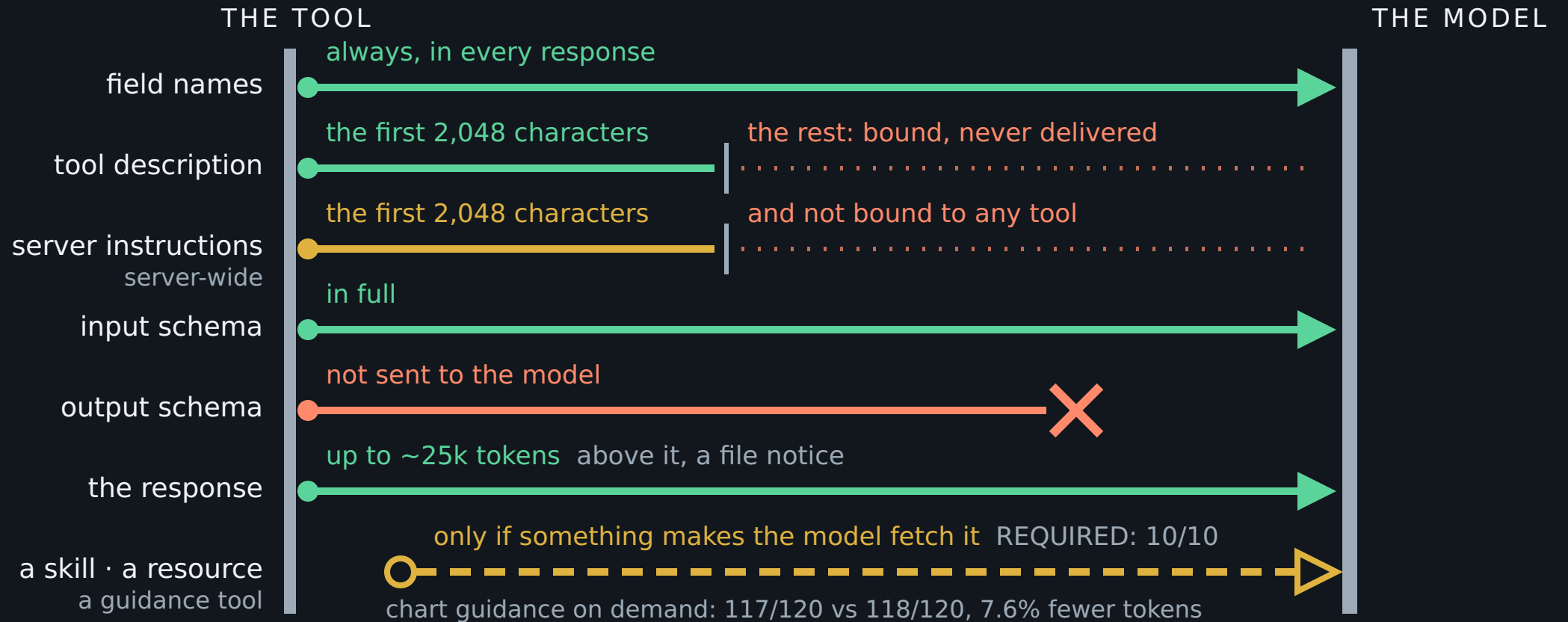
The instruction only fires if the model knows **when** it applies, and that is the fact. **Direction needs knowledge.**

my own rich server the same mistake, as an alert: **0/20** · alert removed: still **0/10**, the right sentence was in there

A surface that can fix 59 answers **can break 59.**

Bound is not the same as delivered

what reaches the model on **Claude Code**, the strictest client measured



same sentence, Haiku past the cut 1/20 · description, char 325: 20/20 · response: 20/20

Check what the model received, not what the server sent.

Every client drops something different

	Claude Code	Cowork	claude.ai chat	ChatGPT	Codex
tool description	first 2,048	first 4,096	in full	in full	in full, after a search
server instructions	first 2,048	in full	none	first 512	in full, on every tool
input schema	in full	in full	in full	in full	describes dropped past 5,000
output schema	none	none	none	not reliably	yes

Design for the strictest: what must hold before the call in the first 2,048 · instructions under 512, and repeated · each input schema under 5,000 · never only the output schema

my own best server failed too, until one table schema went from **6,155 to 4,861** characters

snapshot, September 2026 · n=1 per client · may have changed since · chat and Cowork have merged

Of 30 public MCP servers, **only 10 arrive whole on every client.**

Knowledge as code: one rule from best

all three findings in one rule · shortened from src/domain/best-building-rules.ts

```
{
  id: 'bp.overheating',
  kind: 'verdict',

  applies: (c) => isValue(c.d.overheatingRisk),

  render: (c) => `Overheating risk: ${level},
    indicator ${indicator} (T0juli/GT0, unitless;
    not °C, not hours). Above 1.5: significant.` ,

  provenance: '2026-09-22 · the bare index was read
    as hours or °C · evals/results/...verification.json',
}
```

when

only if the field is in the response, with a value

what the model sees

one line, in the response, whole

why

date, reason, the eval behind it

never sent: for whoever improves it, human or agent

measured: agents cited it **16 of 16**

what the model receives with the record at 3.59

```
"alerts": ["Overheating risk: SIGNIFICANT, indicator 3.59 (T0juli/GT0, unitless; not °C, not hours).
Thresholds: 0 none, up to 1.5 minor, above 1.5 significant."]
```

The knowledge lives in code. The model gets one line, when it applies.

From UX to AX

UX

user experience

AX

agent experience

who	a person	an agent
meets	your screens and flows	your agent interface
can do	features	tailor-made tools
understands	labels, help text, training	knowledge and direction, just in time
when it fails	they call support	it guesses

a thesis for the coming years

We learned to design for users. Now we design for agents.

Domain knowledge belongs in the MCP server.

Yes. Built as a rich domain.

Knowledge, direction and tailor-made tools.
Delivered when the agent needs them.

An extracted demo repo, showing part of this:

a skill that runs the loop on **your** server
a check of what each client drops from it
the practitioner paper
the reference implementation, and the runs behind it
thin, rich and best: three MCP servers on the same public API
these slides, as a PDF

`davidgolverdingen.nl`

`linkedin.com/in/davidgolverdingen`



`github.com/DaveGold/
mcp-metadata-demo`